

NATIONAL UNIVERSITY OF SAN AGUSTÍN

FACULTY OF PRODUCTION AND SERVICES
ENGINEERING

PROFESSIONAL SCHOOL OF SYSTEMS ENGINEERING



INSTALLATION AND DEPLOYMENT GUIDE: DISTRIBUTED RPC SYSTEM (FLASK + RABBITMQ)

Professor: Jesús Martín Silva Fernández

Course: Distributed Systems

Date: May 06, 2026

Team Members:

Larico Rodriguez, Bryan Fernando
Maldonado Vilca, Victor Gonzalo
Quispe Huaman, Rodrigo Ferdinand
Salas Aguilar, Juan Victor

Arequipa - Peru

Contents

1	System Description	2
2	System Architecture	2
3	Prerequisites	3
4	Technologies Used	3
5	Local Installation and Configuration	3
5.1	Clone the Repository	3
5.2	Development Environment	3
6	RabbitMQ Configuration (CloudAMQP)	4
7	Deployment on Render.com	4
7.1	Service 1: Flask Web Service	4
7.2	Service 2: Worker Service	4
8	Troubleshooting	4
9	Conclusions	4
10	References	5

1. System Description

This project consists of the implementation of the **Remote Procedure Call (RPC)** pattern using **Flask** as the HTTP client/server framework and **RabbitMQ** (through CloudAMQP) as the message broker.

The system is designed to be deployed on **Render.com** using two independent components that communicate exclusively through message queues.

The RPC pattern allows a program to invoke remote procedures as if they were local functions. In this project, RabbitMQ acts as the messaging intermediary, enabling decoupling between the client and the worker to improve scalability and fault tolerance.

2. System Architecture

The data flow follows an asynchronous model where the broker acts as an intermediary:

1. The user sends a request to the **Flask Server**.
2. Flask publishes the task to the `rpc_queue` in **CloudAMQP**.
3. The **Worker** consumes the message, processes the information (uppercase conversion), and returns the result.
4. The client retrieves the result and displays it to the user.

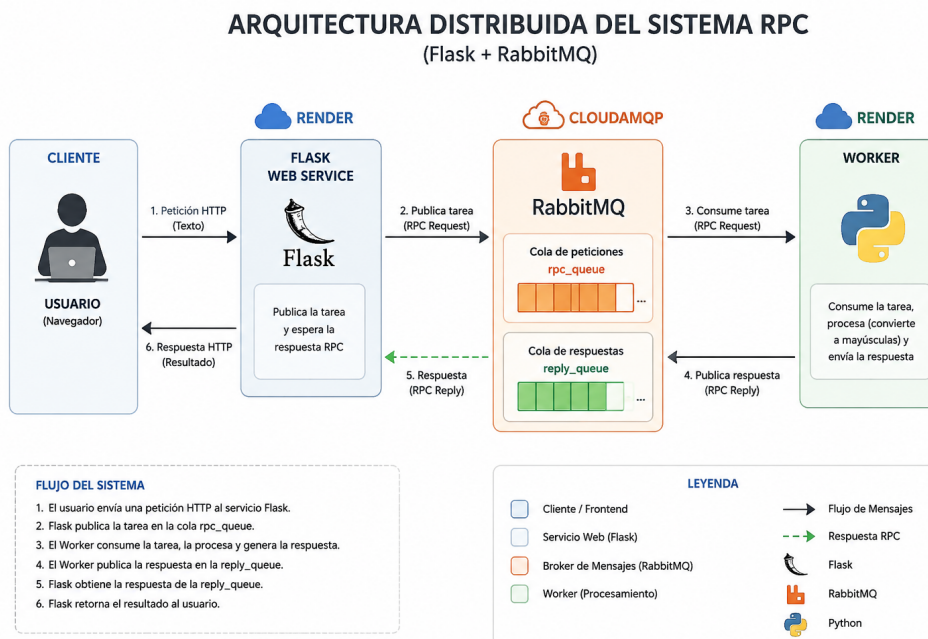


Figure 1: Distributed architecture of the RPC system

3. Prerequisites

To run the system correctly, ensure you have the following:

- Python 3.9 or higher.
- An active account on [Render.com](https://render.com).
- A free Little Lemur instance on [CloudAMQP](https://cloudamqp.com).
- Git installed on your system.

4. Technologies Used

- Python 3.11
- Flask
- RabbitMQ
- CloudAMQP
- Render.com
- Gunicorn
- AMQPStorm

5. Local Installation and Configuration

5.1. Clone the Repository

```
1 git clone https://github.com/Victor-Gonzalo-Maldonado-Vilca/Sistemas_Distribuidos_RPC.git
2 cd Sistemas_Distribuidos_RPC
```

5.2. Development Environment

Create a virtual environment to isolate dependencies:

```
1 python -m venv venv
2
3 # Activate on Linux/Mac
4 source venv/bin/activate
5
6 # Activate on Windows
7 venv\Scripts\activate
```

Install the required packages:

```
1 pip install -r requirements.txt
```

6. RabbitMQ Configuration (CloudAMQP)

1. Access your CloudAMQP dashboard.
2. Copy the **AMQP URL**.
3. Replace the value of the `URL_NUBE` variable in the files:
 - `worker.py`
 - `amqpstorm_threaded_rpc_client.py`

7. Deployment on Render.com

For cloud deployment, it is essential to configure two separate services to support the distributed architecture.

7.1. Service 1: Flask Web Service

- **Build Command:** `pip install -r requirements.txt`
- **Start Command:** `gunicorn amqpstorm_threaded_rpc_client:app`

7.2. Service 2: Worker Service

Due to Render free-tier limitations, the Worker was deployed as a **Web Service** configured to listen on port 10000 using a lightweight dummy HTTP server to prevent Render from stopping the process.

- **Start Command:** `python worker.py`

8. Troubleshooting

- **Timeout Error:** If a timeout occurs, verify that the Worker service is in the **Live** state. Render free-tier services may enter sleep mode after 15 minutes of inactivity.
- **Connection Error:** Ensure that the CloudAMQP URL is correct and does not contain invalid characters.

Render free-tier services automatically enter sleep mode after approximately 15 minutes of inactivity.

9. Conclusions

- A distributed RPC-based system was successfully implemented.
- RabbitMQ enabled decoupled communication between the client and the worker service.

- Deployment on Render demonstrated the feasibility of cloud-based distributed architectures.
- The system can be easily scaled by adding multiple worker instances.

10. References

- RabbitMQ Documentation: <https://www.rabbitmq.com/>
- Flask Documentation: <https://flask.palletsprojects.com/>
- Render Documentation: <https://render.com/docs>
- CloudAMQP Documentation: <https://www.cloudamqp.com/docs/>